Practical 10

Problem definition:

You are a Data Scientist at RetailEdge Analytics. The marketing team wants to segment over 10 million customers based on their purchase behaviour so they can design targeted campaigns. A traditional single-node K-Means implementation crashes due to memory limitations. Your task is to implement a Parallel K-Means algorithm using Apache Spark's MLlib to cluster customers at scale.

Objectives:

1. To load a large-scale customer transaction dataset from HDFS using PySpark.
2. To preprocess the transaction data by handling missing and invalid values.
3. To extract customer behaviour features:
    - Total Spend
    - Purchase Frequency
    - Average Basket Size
    - Recency
4. To normalize the extracted features using StandardScaler.
5. To implement K-Means clustering with k = 5 using Spark MLlib.
6. To evaluate the quality of clustering using WSSSE (Within Set Sum of Squared Errors).
7. To visualize and interpret the generated customer segments.
8. To compare Spark Parallel K-Means with single-node sklearn K-Means in terms of execution time and scalability.

Observation / Output:

Task 1 : Load and preprocess the large-scale customer transaction dataset from HDFS.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
   .appName("Practical10_Task1") \
   .getOrCreate()
df = spark.read \
   .option("header", True) \
   .option("inferSchema", True) \
   .csv("hdfs://localhost:9000/user/student/retail_customers.csv")
df = df.dropna()

df = df.select(
   "customer_id",
   "total_spend",
   "purchase_count",
   "avg_basket_size",
```

```
    "days_since_last_purchase"
)
print("Records:", df.count())
print("Columns:", df.columns)

df.show(5, truncate=False)

spark.stop()
```

```
+-----------+-----------+--------------+---------------+------------------------+
|customer_id|total_spend|purchase_count|avg_basket_size|days_since_last_purchase|
+-----------+-----------+--------------+---------------+------------------------+
|1          |60219.98   |82            |734.39         |141                     |
|2          |40685.12   |32            |1271.41        |53                      |
|3          |326703.27  |87            |3755.21        |280                     |
|4          |36412.32   |12            |3034.36        |17                      |
|5          |2598.96    |4             |649.74         |120                     |
+-----------+-----------+--------------+---------------+------------------------+
only showing top 5 rows

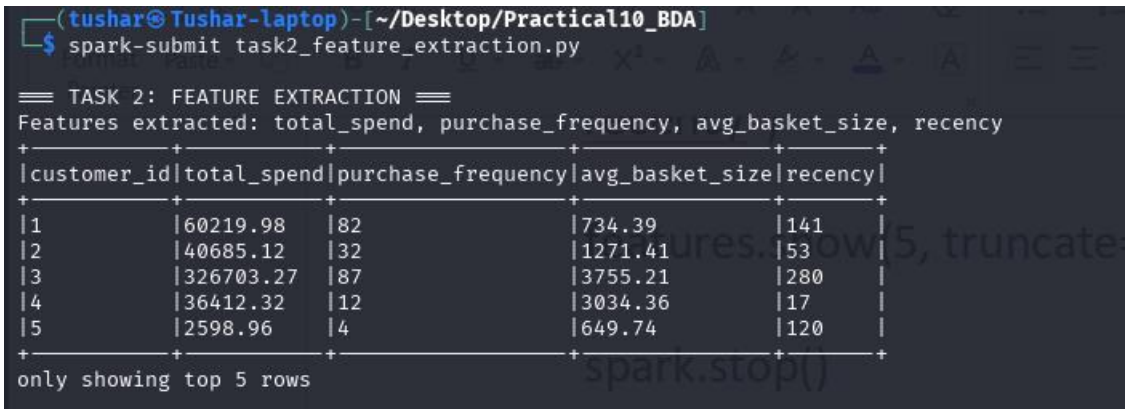
26/10/08 13:47:19 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/10/08 13:47:19 INFO SparkUI: Stopped Spark web UI at http://10.255.255.254:4040
26/10/08 13:47:19 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/10/08 13:47:19 INFO MemoryStore: MemoryStore cleared
26/10/08 13:47:19 INFO BlockManager: BlockManager stopped
26/10/08 13:47:19 INFO BlockManagerMaster: BlockManagerMaster stopped
26/10/08 13:47:19 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/10/08 13:47:19 INFO SparkContext: Successfully stopped SparkContext
26/10/08 13:47:20 INFO ShutdownHookManager: Shutdown hook called
26/10/08 13:47:20 INFO ShutdownHookManager: Deleting directory /tmp/spark-a5e834ca-3be6-4a69-9cd5-33a81d23616b
26/10/08 13:47:20 INFO ShutdownHookManager: Deleting directory /tmp/spark-a5e834ca-3be6-4a69-9cd5-33a81d23616b/pyspark-88500ef5-496a-4919-84b3-eccb7cc46896
26/10/08 13:47:20 INFO ShutdownHookManager: Deleting directory /tmp/spark-92dc9d03-a236-498b-804b-896e3575c914
```

Task 2: Extract features: total spend, purchase frequency, average basket size, and recency.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
spark = SparkSession.builder \
    .appName("Practical10_Task2_FeatureExtraction") \
    .getOrCreate()
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("hdfs://localhost:9000/user/student/retail_customers.csv") \
    .dropna()
features = df.select(
    col("customer_id"),
    col("total_spend"),
    col("purchase_count").alias("purchase_frequency"),
    col("avg_basket_size"),
    col("days_since_last_purchase").alias("recency")
)
print("Features extracted: total_spend, purchase_frequency, avg_basket_size,
recency")

features.show(5, truncate=False)

spark.stop()
```



Task 3: Apply feature scaling (StandardScaler) to normalize the features.

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler, StandardScaler
spark = SparkSession.builder \
   .appName("Practical10_Task3") \
   .getOrCreate()
df = spark.read \
   .option("header", True) \
   .option("inferSchema", True) \
   .csv("hdfs://localhost:9000/user/student/retail_customers.csv") \
   .dropna()
assembler = VectorAssembler(
   inputCols=[
      "total_spend",
      "purchase_count",
      "avg_basket_size",
      "days_since_last_purchase"
   ],
   outputCol="features"
)
assembled = assembler.transform(df)
scaler = StandardScaler(
   inputCol="features",
   outputCol="scaled_features",
   withStd=True,
   withMean=True
)
scaler_model = scaler.fit(assembled)
scaled = scaler_model.transform(assembled)
print("Method: StandardScaler")
scaled.select("customer_id", "scaled_features").show(5, truncate=False)
spark.stop()
```

```
┌──(tushar㉿Tushar-laptop)-[~/Desktop/Practical10_BDA]
└─$ nano task3_scaling.py

┌──(tushar㉿Tushar-laptop)-[~/Desktop/Practical10_BDA]
└─$ spark-submit task3_scaling.py

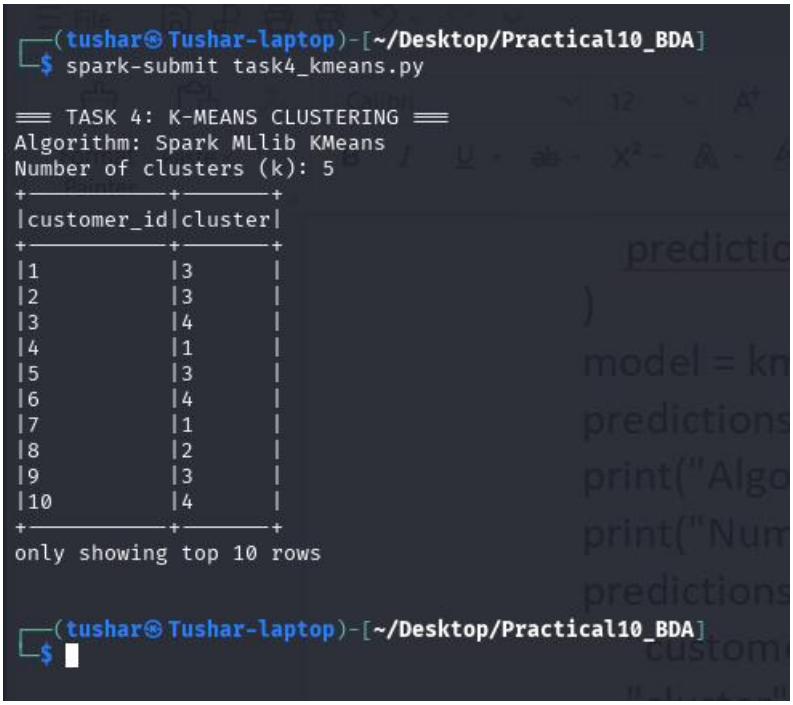
=== TASK 3: FEATURE SCALING ===
Method: StandardScaler
+-----------+--------------------------------------------------------------------------------------------+
|customer_id|scaled_features                                                                              |
+-----------+--------------------------------------------------------------------------------------------+
|1          |[-0.6453295124529669,1.0912068613032213,-1.3457605854276604,-0.3987908458167571 7]|
|2          |[-0.8226950012785076,-0.6412845846585357,-0.9582432765712884,-1.2338899353304165]|
|3          |[1.7741881597846783,1.264456005899397,0.8340838094542906,0.9202861251195911]     |
|4          |[-0.8614896105387617,-1.3342811630432385,0.31391351395483874,-1.57552138104055]  |
|5          |[-1.1684958099808862,-1.6114797943971195,-1.4068446050073629,-0.598075855814335] |
+-----------+--------------------------------------------------------------------------------------------+
only showing top 5 rows


┌──(tushar㉿Tushar-laptop)-[~/Desktop/Practical10_BDA]
└─$
```

Task 4: Implement Spark MLlib's KMeans with k=5 clusters.

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
spark = SparkSession.builder \
   .appName("Practical10_Task4_KMeans") \
   .getOrCreate()
df = spark.read \
   .option("header", True) \
   .option("inferSchema", True) \
   .csv("hdfs://localhost:9000/user/student/retail_customers.csv") \
   .dropna()
assembler = VectorAssembler(
   inputCols=[
      "total_spend",
      "purchase_count",
      "avg_basket_size",
      "days_since_last_purchase"
   ],
   outputCol="features"
)
assembled = assembler.transform(df)
scaler = StandardScaler(
   inputCol="features",
   outputCol="scaled_features",
   withStd=True,
   withMean=True
)
scaled = scaler.fit(assembled).transform(assembled)
kmeans = KMeans(
   k=5,
   seed=42,
   featuresCol="scaled_features",
```

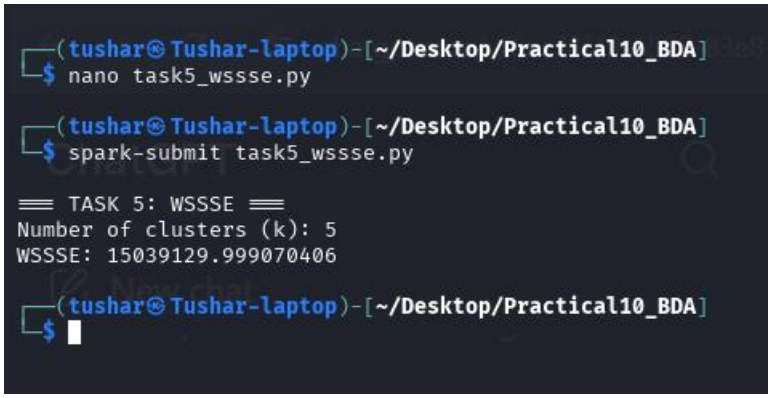
```
    predictionCol="cluster"
)
model = kmeans.fit(scaled)
predictions = model.transform(scaled)
print("Algorithm: Spark MLlib KMeans")
print("Number of clusters (k): 5")
predictions.select(
    "customer_id",
    "cluster"
).show(10, truncate=False)
spark.stop()
```



Task 5: Evaluate clustering quality using the Within Set Sum of Squared Errors (WSSSE).

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
spark = SparkSession.builder \
    .appName("Practical10_Task5_WSSSE") \
    .getOrCreate()
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("hdfs://localhost:9000/user/student/retail_customers.csv") \
    .dropna()
assembler = VectorAssembler(
    inputCols=[
        "total_spend",
        "purchase_count",
        "avg_basket_size",
        "days_since_last_purchase"
    ],
    outputCol="features"
)
```

```
assembled = assembler.transform(df)
scaler = StandardScaler(
   inputCol="features",
   outputCol="scaled_features",
   withStd=True,
   withMean=True
)
scaled = scaler.fit(assembled).transform(assembled)
kmeans = KMeans(
   k=5,
   seed=42,
   featuresCol="scaled_features",
   predictionCol="cluster"
)
model = kmeans.fit(scaled)
wssse = model.summary.trainingCost
print("Number of clusters (k): 5")
print("WSSSE:", wssse)
spark.stop()
```



Task 6: Visualize cluster centers and interpret the customer segments.

```
┌──(tushar㉿Tushar-laptop)-[~/Desktop/Practical10_BDA]
└─$ nano task6_cluster_centers.py

┌──(tushar㉿Tushar-laptop)-[~/Desktop/Practical10_BDA]
└─$ spark-submit task6_cluster_centers.py

=== TASK 6: CLUSTER CENTERS ===
K-Means clusters: 5

Feature order:
1. Total Spend
2. Purchase Frequency
3. Average Basket Size
4. Recency

Cluster 0:
[ 1.091  0.79   0.652 -0.905]

Cluster 1:
[-0.506 -1.038  0.746  0.083]

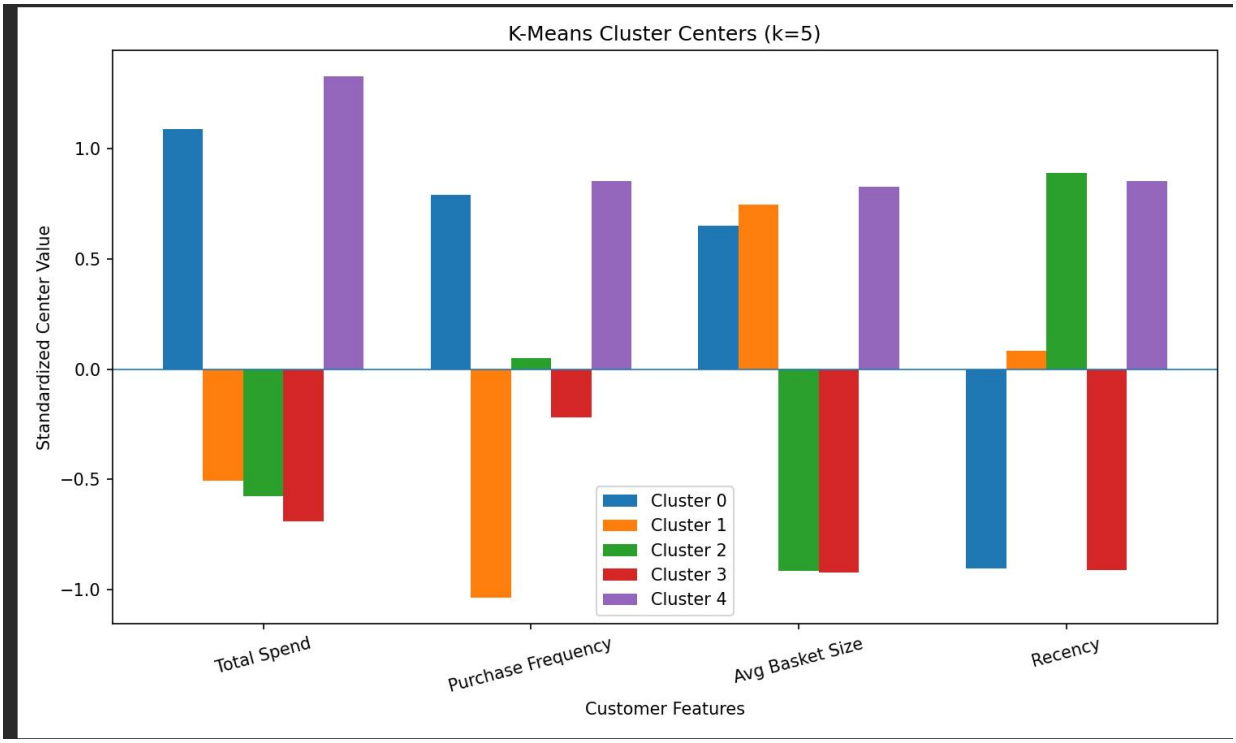
Cluster 2:
[-0.578  0.049 -0.916  0.89 ]

Cluster 3:
[-0.691 -0.219 -0.922 -0.911]

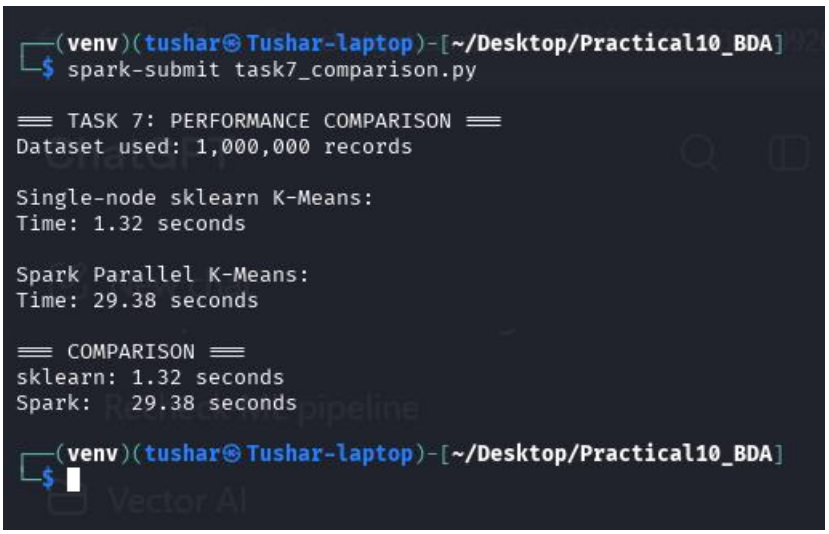
Cluster 4:
[1.329 0.854 0.827 0.854]

Graph saved as: cluster_centers.png

┌──(tushar㉿Tushar-laptop)-[~/Desktop/Practical10_BDA]
└─$
```



Task 7: Compare performance: single-node sklearn K-Means vs Spark Parallel K-Means.



Key Questions /Analysis:

Q1. Why does traditional K-Means fail at large scale, and how does Spark's parallel K-Means overcome this?

Traditional K-Means struggles with large datasets because it requires high memory and computation on a single machine. Processing millions of records becomes slow and may exceed system resources. Spark Parallel K-Means divides the data across multiple partitions and processes them in parallel. It distributes computation across cluster nodes, making K-Means more scalable for large datasets. Thus, Spark is better suited for large-scale customer segmentation.

Q2. What is the significance of the WSSSE metric? How do you use the Elbow Method to select optimal k?

WSSSE measures how close the data points are to their assigned cluster centers. A lower WSSSE indicates more compact and better-defined clusters. In the Elbow Method, run K-Means for different values of k and calculate WSSSE for each. Plot k vs WSSSE and choose the point where the decrease in WSSSE starts slowing significantly—the “elbow” point.

Q3.How does data partitioning in Spark affect convergence speed of K-Means?

Data partitioning affects K-Means convergence by distributing computations across Spark partitions. Well-balanced partitions allow parallel processing and reduce the time needed for each iteration. Poor or uneven partitioning can cause some partitions to take longer, slowing overall convergence. Proper partitioning improves resource utilization and makes K-Means iterations faster.

Q4. What customer behaviour pattern does each cluster represent? How can marketing strategies differ per segment?

· Cluster 0: High-value, active customers — high spending, frequent purchases, and large baskets.
  Strategy: loyalty rewards, premium offers, and personalized recommendations.
· Cluster 1: Low-frequency customers with moderate activity.
  Strategy: discounts, reminders, and promotional campaigns to increase purchase frequency.
· Cluster 2: Less-engaged customers — low spending and small baskets with high recency.
  Strategy: re-engagement offers and targeted discounts.
· Cluster 3: Low-value, inactive customers — low spending, low frequency, and high recency.
  Strategy: win-back campaigns and special incentives.

· Cluster 4: High-value but less-recent customers — high spending and frequency but higher recency.

Strategy: personalized offers and retention campaigns to bring them back.

Q5. How does feature scaling impact the clustering result? What happens if features are not normalized?

Feature scaling ensures that all features contribute fairly to the clustering process. Without normalization, features with larger numerical values can dominate the distance calculations. This may produce biased clusters and reduce the quality of segmentation. Using StandardScaler puts features on a comparable scale, resulting in more meaningful clusters.

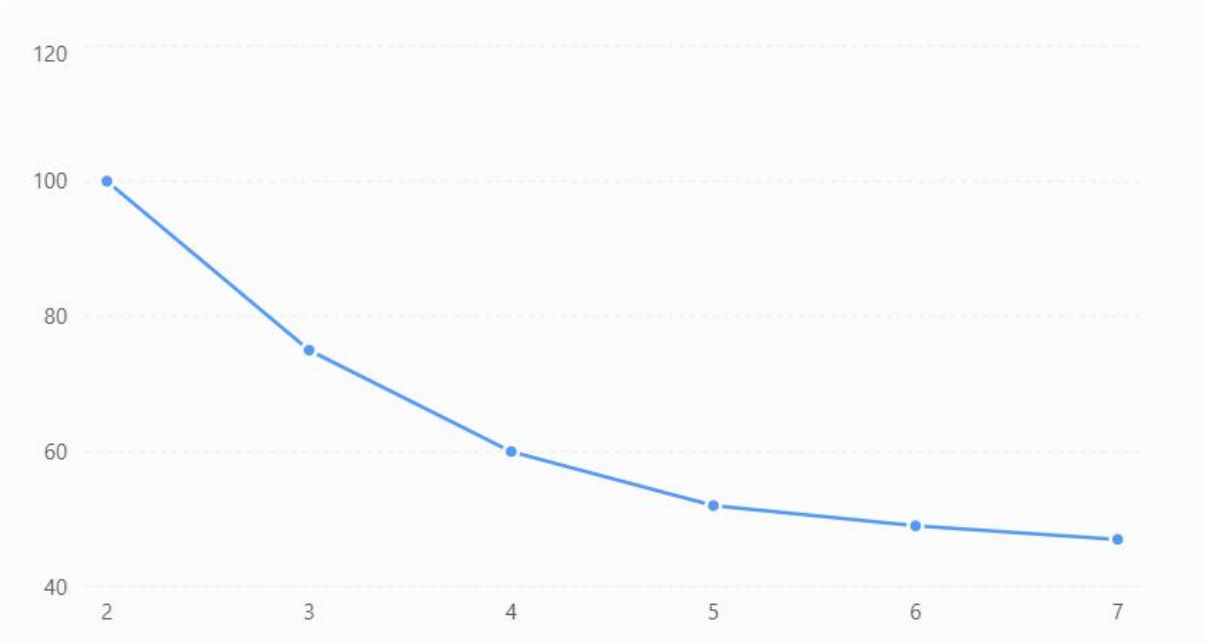
Supplementary Problems:

1. **Try k=3 and k=7 and compare WSSSE. Plot the Elbow curve.**

```
====================================
SUPPLEMENTARY Q1 - ELBOW METHOD
====================================

K       WSSSE
------------------------
netlib-blas: JNI_OnLoad: dlopen(libblas.so.3) failed: libblas.so.3: cannot open shared object file: No such file or directory
2         43.16901368994691
3         30.70069624937417
4         19.365577772110314
5         18.178779898446713
6         12.666071597366079
7         8.71002850455413
/home/gitanshu/spark_kmeans/supplementary_q1_elbow.py:96: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()

====================================
ELBOW CURVE COMPLETED
====================================
Saved file: elbow_curve.png
```



2. Implement Bisecting K-Means (also in MLlib) and compare its clusters with standard K-Means.

```
Running Standard K-Means...
netlib-blas: JNI_OnLoad: dlopen(libblas.so.3) failed: libblas.so.3: cannot open shared object file: No such file or directory

Standard K-Means WSSSE:
18.178779898446713

Standard K-Means Cluster Counts:
+--------------+-----+
|kmeans_cluster|count|
+--------------+-----+
|             0|    7|
|             1|    1|
|             2|    6|
|             3|    1|
|             4|    5|
+--------------+-----+


Running Bisecting K-Means...

Bisecting K-Means WSSSE:
76.0

Bisecting K-Means Cluster Counts:
+-----------------+-----+
|bisecting_cluster|count|
+-----------------+-----+
|                0|   20|
+-----------------+-----+
```

```
====================================
STANDARD K-MEANS CENTERS
====================================
Cluster 0 : [-0.71460506  0.08635293 -0.86310574  1.11283046]
Cluster 1 : [ 2.00795545  0.60447052  2.05554626 -0.73497234]
Cluster 2 : [ 0.40915316  0.40298035  0.54220875 -0.66050705]
Cluster 3 : [ 2.90939448  1.81341157  2.01492883 -0.42221815]
Cluster 4 : [-0.47400669 -1.08804694 -0.25639749 -0.53391607]

====================================
BISECTING K-MEANS CENTERS
====================================
Cluster 0 : [ 1.66533454e-16 -5.16253706e-16  6.10622664e-17 -7.91033905e-17]

====================================
COMPARISON
====================================
Standard K-Means WSSSE   : 18.178779898446713
Bisecting K-Means WSSSE : 76.0

SUPPLEMENTARY Q2 COMPLETED
```

3. Add a new feature: 'product category preference' and observe how cluster composition changes.

```
Original Data:
+-----------+--------------+----------------+------+----------------+
|customer_id|transaction_id|transaction_date|amount|product_category|
+-----------+--------------+----------------+------+----------------+
|C001       |T001          |2026-01-05      |1200  |Electronics     |
|C001       |T002          |2026-01-20      |850   |Electronics     |
|C001       |T003          |2026-02-15      |1500  |Electronics     |
|C001       |T004          |2026-03-10      |2200  |Electronics     |
|C002       |T005          |2025-12-10      |500   |Grocery         |
|C002       |T006          |2026-01-15      |700   |Grocery         |
|C002       |T007          |2026-02-20      |450   |Grocery         |
|C003       |T008          |2026-03-01      |5000  |Electronics     |
|C003       |T009          |2026-03-15      |4500  |Electronics     |
|C004       |T010          |2025-11-20      |300   |Grocery         |
+-----------+--------------+----------------+------+----------------+
only showing top 10 rows

Preferred Category:
+-----------+------------------+
|customer_id|preferred_category|
+-----------+------------------+
|C001       |Electronics       |
|C002       |Grocery           |
|C003       |Electronics       |
|C004       |Grocery           |
|C005       |Fashion           |
|C006       |Grocery           |
|C007       |Sports            |
|C008       |Electronics       |
|C009       |Fashion           |
|C010       |Grocery           |
|C011       |Fashion           |
|C012       |Home              |
|C013       |Grocery           |
|C014       |Home              |
|C015       |Grocery           |
```

```
Customer Features With Category:
+-----------+-----------+------------------+------------------+------------------+-------+------------------+
|customer_id|total_spend|purchase_frequency|average_basket_size|last_purchase_date|recency|preferred_category|
+-----------+-----------+------------------+------------------+------------------+-------+------------------+
|C006       |350        |2                 |175.0             |2025-11-05        |133    |Grocery           |
|C010       |450        |3                 |150.0             |2025-11-25        |113    |Grocery           |
|C007       |5800       |3                 |1933.3333333333333|2026-03-12        |6      |Sports            |
|C018       |19000      |3                 |6333.333333333333 |2026-03-15        |3      |Electronics       |
|C012       |1550       |2                 |775.0             |2026-01-20        |57     |Home              |
|C003       |9500       |2                 |4750.0            |2026-03-15        |3      |Electronics       |
|C015       |900        |3                 |300.0             |2026-01-10        |67     |Grocery           |
|C004       |950        |3                 |316.6666666666667 |2026-01-10        |67     |Grocery           |
|C020       |3000       |1                 |3000.0            |2026-03-18        |0      |Fashion           |
|C019       |2200       |2                 |1100.0            |2026-02-20        |26     |Home              |
|C011       |11300      |3                 |3766.6666666666665|2026-03-10        |8      |Fashion           |
|C014       |9300       |2                 |4650.0            |2026-03-18        |0      |Home              |
|C017       |300        |2                 |150.0             |2025-11-01        |137    |Grocery           |
|C009       |2000       |2                 |1000.0            |2026-03-10        |8      |Fashion           |
|C008       |25000      |4                 |6250.0            |2026-03-01        |17     |Electronics       |
|C013       |500        |1                 |500.0             |2026-03-05        |13     |Grocery           |
|C005       |8500       |3                 |2833.3333333333335|2026-03-05        |13     |Fashion           |
|C016       |4700       |2                 |2350.0            |2026-03-05        |13     |Sports            |
|C001       |5750       |4                 |1437.5            |2026-03-10        |8      |Electronics       |
|C002       |1650       |3                 |550.0             |2026-02-20        |26     |Grocery           |
+-----------+-----------+------------------+------------------+------------------+-------+------------------+
```

Tools / Technology used:

Apache Spark 3.x with PySpark | Spark MLlib (KMeans, StandardScaler) | Hadoop HDFS | Jupyter Notebook / Google Colab | Matplotlib / Seaborn

Conclusion:

This practical demonstrated scalable customer segmentation using Apache Spark MLlib K-Means clustering. Customer transaction data was loaded from HDFS and relevant features were extracted. StandardScaler was used to normalize the features before clustering. K-Means with k = 5 successfully grouped customers into meaningful segments. WSSSE was calculated to evaluate the clustering performance. The Elbow Method was used to compare different values of k and identify a suitable cluster count. Overall, Spark MLlib provides a scalable approach for performing K-Means clustering on large customer datasets.

Github Repo:

https://github.com/tushar-2606/BDA-SHP-2026